

A PROJECT REPORT ON

Enhancing Low-Resource NLP Using Multitask Learning

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Abhay Shanbhag
Amogh Thakurdesai
Ridhima Sinare
Suramya Jadhav

Exam No: B400050590
Exam No: B400050611
Exam No: B400050601
Exam No: B400050421



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
Enhancing Low-Resource NLP Using Multitask Learning

Submitted by

Abhay Shanbhag

Exam No: B400050590

Amogh Thakurdesai

Exam No: B400050611

Ridhima Sinare

Exam No: B400050601

Suramya Jadhav

Exam No: B400050421

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. Sheetal Sonawane** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. Sheetal Sonawane
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe

Principal

Pune Institute of Computer Technology

Place: Pune

Date:

ACKNOWLEDGEMENT

*It gives us great pleasure to present the project report on ‘**Enhancing Low-Resource NLP Using Multitask Learning**’.*

*We would like to take this opportunity to thank **Dr. Sheetal Sonawane** for providing us with all the help and guidance we needed. We are truly grateful for her kind support. Her valuable suggestions were immensely helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of the Computer Engineering Department, PICT, for his indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support. Finally, we extend our special thanks to our honorable **Dr. P. T. Kulkarni**, Director, PICT, for his continued support and valuable inputs during the ideation and implementation phases of the project.*

*Lastly, we offer our heartfelt thanks to **Mr. Raviraj Joshi**, L3Cube, and all faculty members for their wholehearted cooperation in the completion of this report. We also thank our laboratory assistants for their valuable help in the laboratory.*

Abhay Shanbhag
Amogh Thakurdesai
Ridhima Sinare
Suramya Jadhav
(B.E. Computer Engg.)

ABSTRACT

Multi-task learning (MTL) has garnered substantial attention in the field of Natural Language Processing (NLP) for its ability to improve performance by jointly learning multiple tasks. By leveraging the interrelatedness of tasks, MTL can mitigate issues like overfitting and data scarcity, which are particularly relevant in low-resource languages such as Marathi. This survey reviews recent advances in MTL, focusing on its application to NLP tasks in low-resource languages. We examine various MTL training paradigms, including joint training and task-specific optimization, and explore diverse model architectures such as parallel, hierarchical, and modular frameworks. Additionally, we analyze how MTL approaches have been adapted for Marathi and other low-resource languages by integrating domain-specific features and addressing linguistic challenges. Finally, we discuss the potential of MTL in advancing NLP for low-resource languages, identifying challenges such as dataset limitations, task balancing, and cross-lingual transfer, while outlining future research directions aimed at further enhancing language understanding and task-specific outcomes in this area.

Contents

1	Introduction	8
1.1	Overview	9
1.2	Motivation	10
1.3	Problem Definition and Objectives	10
1.4	Project Scope & Limitations	11
1.5	Methodologies of Problem Solving	12
2	Literature Survey	13
2.1	Multitask Learning in High-Resource Languages (HRLs)	14
2.2	Multitask Learning for Low-Resource Languages (LRLs)	15
3	Software Requirements Specification	17
3.1	Introduction	18
3.1.1	Project Scope	18
3.1.2	Use Cases and Characteristics	18
3.1.3	Assumptions and Dependencies	19
3.2	Functional Requirements	20
3.3	Non-Functional Requirements	20
3.3.1	Performance Requirements	20
3.3.2	Safety Requirements	21
3.3.3	Security Requirements	21
3.4	System Requirements	21
3.4.1	Database Requirements	21
3.4.2	Software Requirements	22
3.4.3	Hardware Requirements	22
3.5	Analysis Models: SDLC Model	22
3.5.1	Introduction to Agile SDLC	22

3.5.2	Phases in Agile SDLC for Our Project	22
3.5.3	Collaboration Tools	23
4	System Design	24
4.1	System Architecture	25
4.2	UML Class Diagrams	26
4.3	UML Use-Case Diagrams	27
4.4	UML Activity Diagrams	28
4.5	Data Flow Diagram	29
4.6	Sequence Diagram	30
4.7	Component Diagram	31
4.8	Deployment Diagram	32
5	Project Plan	33
5.1	Project Estimate	34
5.1.1	Reconciled Estimates	34
5.1.2	Project Resources	35
5.2	Risk Management	37
5.2.1	Risk Identification	37
5.2.2	Risk Analysis	37
5.2.3	Overview of Risk Mitigation, Monitoring, Management	38
5.3	Project Schedule	40
5.3.1	Project Task Set	40
5.3.2	Task Network	41
5.3.3	Timeline Chart	41
5.4	Team Organization	42
5.4.1	Team structure	42
5.4.2	Management reporting and communication	43
6	Project Implementation	45
6.1	Overview of Project Modules	46
6.2	Tools and Technologies Used	47
6.3	Algorithm Details	47

7	Software Testing	49
7.1	Type of Testing	50
7.2	Test Cases	51
8	Results	54
8.1	Test and Evaluation Results	55
8.2	Screen Shots	58
9	Conclusions	59
9.1	Conclusions	60
9.2	Future Work	61
9.3	Applications	61
10	Appendix	63
11	References	70

List of Figures

4.1	System Architecture	25
4.2	UML Class Diagram	26
4.3	UML Use-Case Diagram	27
4.4	UML Activity Diagram	28
4.5	Data Flow Diagram	29
4.6	Sequence Diagram	30
4.7	Component Diagram	31
4.8	Deployment Diagram	32
8.1	UI for Multi Task Learning Framework	58
10.1	Submission Details	67
10.2	INC Certificates	68
10.3	Plagiarism Report	69

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

This study explores the application of Multi-task Learning (MTL) to improve the performance of Natural Language Processing (NLP) models, particularly focusing on low-resource languages, with Marathi as a primary case study. MTL allows a model to learn multiple related tasks simultaneously, enhancing its ability to generalize and improving efficiency, particularly when labeled data is scarce. Traditional NLP models often treat each task independently, which can be inefficient, especially for languages like Marathi that have limited resources for training data. This research demonstrates the potential of MTL in addressing these limitations by leveraging shared representations across different tasks.

The primary objective of this study is to propose an MTL pipeline that integrates BERT embeddings for multitask classification on a Marathi Classification Dataset. The dataset includes tasks such as Sentiment Analysis, Hate Speech Detection, and News Classification, each with different types of content and structure. The goal is to assess how training models across multiple tasks—simultaneously—can benefit performance, especially when considering the low-resource nature of Marathi data. In this work, we also establish the importance of task-relatedness in selecting primary and auxiliary tasks, an aspect critical to the success of the MTL approach. The study evaluates various combinations of tasks, providing a comprehensive analysis of how different configurations impact the model’s ability to generalize across multiple domains.

This report provides an in-depth analysis of the pipeline design, dataset characteristics, and experimental setup, concluding with an evaluation of the model’s performance through both single-task learning (STL) and multitask learning (MTL) configurations. The findings highlight the benefits and trade-offs of MTL for low-resource languages like Marathi and suggest practical approaches for future work in NLP under similar resource constraints.

1.2 MOTIVATION

Limited labeled data is a problem for many NLP jobs, particularly in low-resource languages and domains. A viable remedy is multitask learning, which enhances performance by utilizing data from related tasks. Models can become more flexible and perform better on unknown data or in new domains by learning from a variety of tasks. By eliminating the need to train distinct models for every job, multitask learning may result in more effective development and implementation. Numerous NLP applications, including sentiment analysis, machine translation, and question answering, could benefit from multitask learning. The study can lead to new opportunities for language technology and help improve NLP methods, especially in low-resource environments. By improving NLP performance in low-resource languages, the project can help address language inequality and ensure that language technology benefits a wider range of communities.

1.3 PROBLEM DEFINITION AND OBJECTIVES

The objective of this work is to design a robust multitask learning framework that enhances natural language processing (NLP) performance in low-resource environments. The specific goals of the project are outlined below:

1. To develop a robust and efficient multitask learning framework that can significantly improve the performance of NLP tasks in low-resource settings.
2. To explore the impact of task relatedness and data distribution on the performance of multitask learning models.
3. To compare the performance of the proposed multitask learning approach with existing single-task and transfer learning methods.

1.4 PROJECT SCOPE & LIMITATIONS

1) Project Scope

1. **Multitask Learning Pipeline:** Focus on applying multitask learning (MTL) with BERT embeddings to Marathi text classification tasks such as Sentiment Analysis, Hate Speech Detection, and News Classification.
2. **Task Grouping:** Analyze task-relatedness to identify and optimize effective combinations of tasks for improved model performance, particularly in low-resource settings.
3. **Low-Resource Language Focus:** Investigate the benefits of MTL for Marathi, a low-resource language, by leveraging shared learning across related tasks to improve generalization and efficiency.

2) Limitations

1. **Dataset Limitations:** The quality and quantity of available Marathi datasets are limited, which can restrict model generalization and reduce overall performance.
2. **Task Selection Challenges:** Poorly related tasks may lead to negative transfer or task interference, reducing the effectiveness of multitask learning.
3. **Training Complexity:** Multitask learning increases training complexity and computational demands, requiring more resources and careful balancing of task-specific losses.

1.5 METHODOLOGIES OF PROBLEM SOLVING

The methodology adopted for this project centers around designing an effective multitask learning (MTL) framework to enhance the performance of Marathi text classification tasks in low-resource settings. The primary approach involves applying multitask learning, where a single model is simultaneously trained on multiple related tasks. By learning shared representations across these tasks, the model is able to improve generalization and performance, which is particularly beneficial in scenarios with limited labeled data.

A BERT-based model architecture forms the foundation of the system. Pre-trained BERT embeddings are fine-tuned for various Marathi text classification tasks, including Sentiment Analysis, Hate Speech Detection, and News Classification. The [CLS] token representation is utilized for each downstream task, ensuring that the fine-tuned model captures task-specific features effectively while benefiting from BERT's strong contextual understanding.

An important aspect of the methodology involves evaluating different task combinations to analyze their impact on overall model performance. Configurations involving two, three, four, and five tasks are explored to identify the optimal groupings that yield the best results. This evaluation helps in understanding the degree of task relatedness and its influence on the multitask learning setup. Overall, the methodology aims to leverage the advantages of multitask learning to enhance text classification performance in Marathi, a language with limited computational resources and annotated datasets.

CHAPTER 2

LITERATURE SURVEY

2.1 MULTITASK LEARNING IN HIGH-RESOURCE LANGUAGES (HRLS)

Due to the availability of abundant resources and annotated datasets, multitask learning (MTL) has been extensively applied in Natural Language Processing (NLP), especially for English. Researchers have utilized MTL to enhance performance in areas such as speech recognition, sentiment analysis, and machine translation. This study highlights key architectures, methodologies, and findings from significant investigations.

Wang et al. (2021) demonstrated that shared encoder-decoder architectures for English, a high-resource language, can significantly improve multilingual neural machine translation tasks. Li et al. (2021) assessed MTL in deep neural networks for tasks such as text classification, question answering, and sentiment analysis, emphasizing the role of parameter sharing to improve learning efficiency and reduce overfitting.

In a comprehensive survey, Zhang et al. (2023) categorized MTL models into hard and soft parameter sharing frameworks, emphasizing the importance of task relatedness for tasks like dependency parsing, Named Entity Recognition (NER), and part-of-speech tagging. Worsham and Kalita (2020) discussed potential future directions, advocating modular and hierarchical architectures to tackle complex challenges like contextual understanding in dialogue systems.

McCann et al. (2018) introduced the NLP Decathlon, demonstrating that a single MTL model trained on ten different English NLP tasks can outperform single-task models and generalize across tasks. Bi et al. (2022) applied BERT-based MTL techniques to news recommendation systems, enhancing user preference modeling and text comprehension by training on tasks such as click-through prediction and sentiment analysis.

Sawhney et al. (2021) utilized MTL to improve emotional recognition in English text, particularly in sensitive domains like disclosures of sexual abuse. Rastogi et al. (2018) demonstrated that parameter sharing between speech recognition and translation

tasks in MTL can reduce error propagation in end-to-end speech translation. Cheng et al. (2022) applied MTL for sequential sentence classification in academic articles, improving tasks like citation intent detection and sentence classification. Huang et al. (2022) showed that MTL enhances English speech processing systems by sharing knowledge across tasks such as text-based sentiment analysis and spoken emotion recognition.

While MTL has clearly led to performance and efficiency improvements across a wide range of English NLP tasks, challenges remain, particularly in task selection, model design, and optimization for complex tasks.

2.2 MULTITASK LEARNING FOR LOW-RESOURCE LANGUAGES (LRLS)

The scarcity of annotated datasets in low-resource languages has spurred growing interest in applying multitask learning (MTL) to languages such as Tamil, Kannada, Malayalam, Marathi, Bengali, Hindi, Indonesian, and Turkish. MTL addresses the challenges of limited data by enabling shared representations across tasks, improving performance and generalization.

Hande et al. (2022) investigated MTL for Dravidian languages (Tamil, Kannada, and Malayalam), focusing on syntactic and semantic tasks such as NER and part-of-speech tagging. Using a hard parameter-sharing strategy, they achieved significant performance gains. Schulz et al. (2018) demonstrated the benefits of using auxiliary tasks like sentiment analysis to improve text classification and argument mining in Bengali and Marathi.

Kumar et al. (2024) showed that parameter sharing between POS tagging and dependency parsing improved model robustness in noisy social media text for Tamil and Malayalam. Ikhwantri et al. (2018) applied MTL to semantic role labeling (SRL) for Hindi and Marathi, integrating related tasks such as NER and relation extraction to

reduce dependency on large annotated datasets.

Magooda et al. (2021) enhanced abstractive summarization performance for Bengali and Hindi by sharing representations with auxiliary tasks like language modeling. Sánchez-Cartagena et al. (2021) employed MTL combined with data augmentation strategies to improve neural machine translation quality for low-resource languages like Turkish and Indonesian.

Wang et al. (2020) focused on multilingual neural machine translation for Turkish and Marathi, using English as a pivot language to boost translation accuracy through shared encoder-decoder models. Madhavaraj and Ramakrishnan (2019) applied MTL to improve automated speech recognition (ASR) systems for Tamil, Kannada, and Marathi by incorporating related tasks such as speaker identification and sentiment analysis.

While MTL has proven effective in addressing the challenges faced by low-resource languages and improving tasks like SRL, machine translation, and POS tagging, significant challenges remain. These include careful task selection, architecture design, and the need for further exploration of unsupervised and task-specific learning techniques.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 INTRODUCTION

3.1.1 Project Scope

Objective: The objective of this project is to develop a multitask learning (MTL) framework to enhance the performance of natural language processing (NLP) tasks in low-resource language environments. The project aims to reduce reliance on large labeled datasets by leveraging related tasks to improve model generalization.

Multitask Learning Approach: This project investigates how MTL can benefit NLP by sharing representations across related tasks such as text classification, named entity recognition (NER), and sentiment analysis. In low-resource languages like Marathi, this technique enables the use of auxiliary tasks to improve the performance of primary tasks.

3.1.2 Use Cases and Characteristics

- **Low-Resource Language Processing:** The main use case is to improve NLP for low-resource languages like Marathi. MTL addresses limited labeled data by enhancing tasks such as NER, sentiment analysis, and text classification.
- **Cross-Task Learning for Efficiency:** MTL allows the model to learn from related tasks, eliminating the need for separate models for each task. This boosts efficiency and reduces computation costs in applications like machine translation.
- **Applications in NLP Tasks:**
 - *Machine Translation:* Improves translation quality for low-resource languages by sharing knowledge across tasks.

- *Sentiment Analysis*: Enhanced using knowledge from related tasks like emotion detection and topic classification.
- *Named Entity Recognition (NER)*: Improved through complementary tasks such as part-of-speech tagging and classification.

3.1.3 Assumptions and Dependencies

- **Data Availability:**

- *Assumption*: Sufficient datasets exist for major tasks and auxiliary jobs, even in Marathi.
- *Dependency*: Model performance depends on access to diverse and related datasets.

- **Task Relatedness:**

- *Assumption*: Chosen tasks are related enough to benefit from shared learning.
- *Dependency*: Poorly related tasks may degrade model performance.

- **Computational Resources:**

- *Assumption*: Sufficient GPU-enabled resources are available.
- *Dependency*: Lack of powerful hardware may hinder experimentation.

- **Pre-Trained Models:**

- *Assumption*: Pre-trained models like BERT or FastText are available for Marathi.
- *Dependency*: The availability and quality of pre-trained models will impact performance.

3.2 FUNCTIONAL REQUIREMENTS

- **Data Collection and Preprocessing:** System should support data acquisition from sources like web scraping, datasets, and user inputs. Preprocessing includes augmentation, stop word removal, tokenization, normalization, and handling out-of-vocabulary words.
- **Multitask Learning Framework:** Build an architecture capable of handling multiple NLP tasks simultaneously using shared representations.
- **Model Training and Evaluation:** Implement training loops, optimize hyperparameters, and evaluate performance using metrics like accuracy, F1-score, precision, and recall.
- **Model Fine-Tuning and Optimization:** Support fine-tuning of pre-trained models and ablation studies to understand the effect of auxiliary tasks.

3.3 NON-FUNCTIONAL REQUIREMENTS

3.3.1 Performance Requirements

- **Model Accuracy:** Minimum accuracy of 80% for primary tasks on validation data.
- **Training Time:** Complete model training within 24–48 hours on a standard GPU.
- **Robustness to Noisy Data:** Maintain within 5% accuracy under 10–20% noise in data.
- **Model Generalization:** Performance on unseen data should remain above 75% of training accuracy.

- **Evaluation Metrics:** System should support metrics like precision, recall, F1-score, and task-specific metrics.

3.3.2 Safety Requirements

- **Data Privacy and Security:** Comply with regulations like GDPR and CCPA.
- **Ethical Guidelines Compliance:** Follow AI and NLP ethical guidelines.

3.3.3 Security Requirements

- **User Authentication:** Implement strong user authentication (e.g., MFA).
- **Data Encryption:** Use TLS/SSL in transit and AES-256 at rest.
- **Secure Data Storage:** Ensure backups, disaster recovery, and secure storage.
- **Access Control:** Use role-based access control (RBAC).

3.4 SYSTEM REQUIREMENTS

3.4.1 Database Requirements

- **Data Migration and Integration:** Support CSV, APIs, and external system integration.

3.4.2 Software Requirements

- **Programming Language:** Python is preferred due to rich ML/NLP libraries.
- **ML Frameworks:** Use TensorFlow or PyTorch.
- **NLP Libraries:** Utilize Hugging Face Transformers, spaCy, or NLTK.

3.4.3 Hardware Requirements

- **Computational Power:** GPU-enabled machine (e.g., NVIDIA T4, RTX 3080/3090).
- **CPU:** High-performance multi-core CPU (e.g., Intel i7 or Ryzen 7).
- **RAM:** Minimum 16 GB RAM.

3.5 ANALYSIS MODELS: SDLC MODEL

3.5.1 Introduction to Agile SDLC

Agile SDLC is an iterative model focusing on adaptability, team collaboration, and continuous user feedback. It suits our project due to the evolving nature of multitask learning for low-resource NLP.

3.5.2 Phases in Agile SDLC for Our Project

- **Requirements Gathering:** Define target NLP tasks (e.g., NER, sentiment analysis).

- **Sprint Planning:** Plan 2–4 week sprints with defined deliverables.
- **Design and Prototyping:** Draft architecture and build initial models.
- **Development:** Implement the multitask framework and integrate modules.
- **Testing and Validation:** Test after each sprint using relevant metrics.
- **Deployment:** Deploy after successful testing for evaluation and feedback.
- **Continuous Feedback and Iteration:** Gather and incorporate feedback throughout development.

3.5.3 Collaboration Tools

Use tools like Jira or Trello for task tracking and team coordination. Agile methodology ensures the system remains responsive to the dynamic requirements of multitask learning in low-resource NLP.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

Diagram 4.1 outlines the flow of a multi-task text classification model. The process begins with raw text inputs, which undergo preprocessing to remove stop words, handle emojis, and correct spelling. The cleaned text is then passed through a tokenizer, which uses the tokenizer from the specific embedding models (MahaBERT, MuRIL, IndicBERT, or FastText). These embeddings convert the tokens into numerical vectors. The embeddings are fed into a shared Multilayer Perceptron (MLP) model that contains hidden layers. From there, the model branches into three distinct classification heads for Hate Classification, Sentiment Analysis, and News Classification, with each head using a softmax function to make predictions.

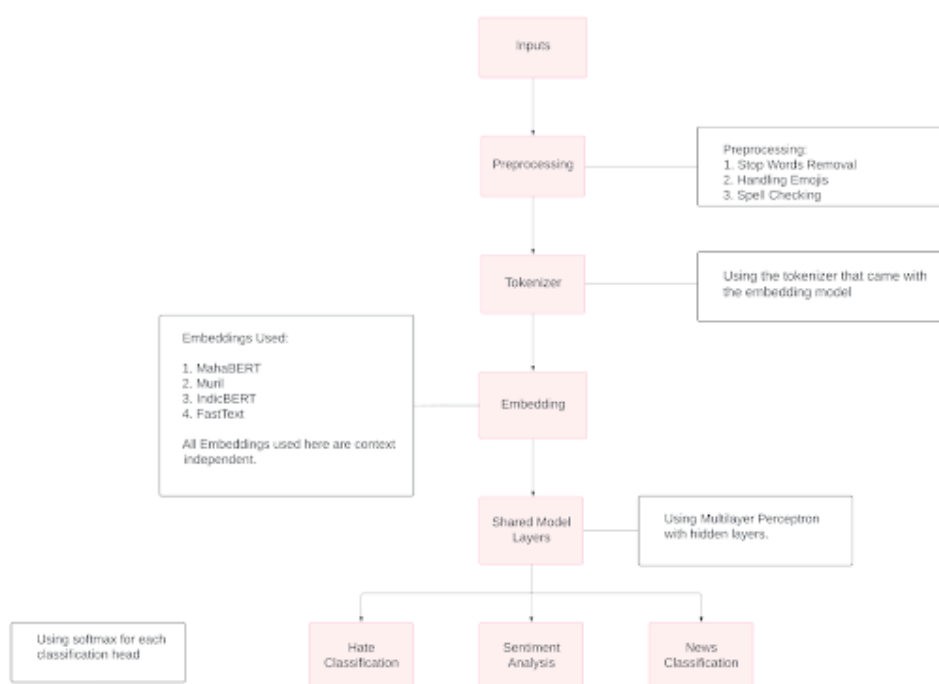


Figure 4.1: System Architecture

4.2 UML CLASS DIAGRAMS

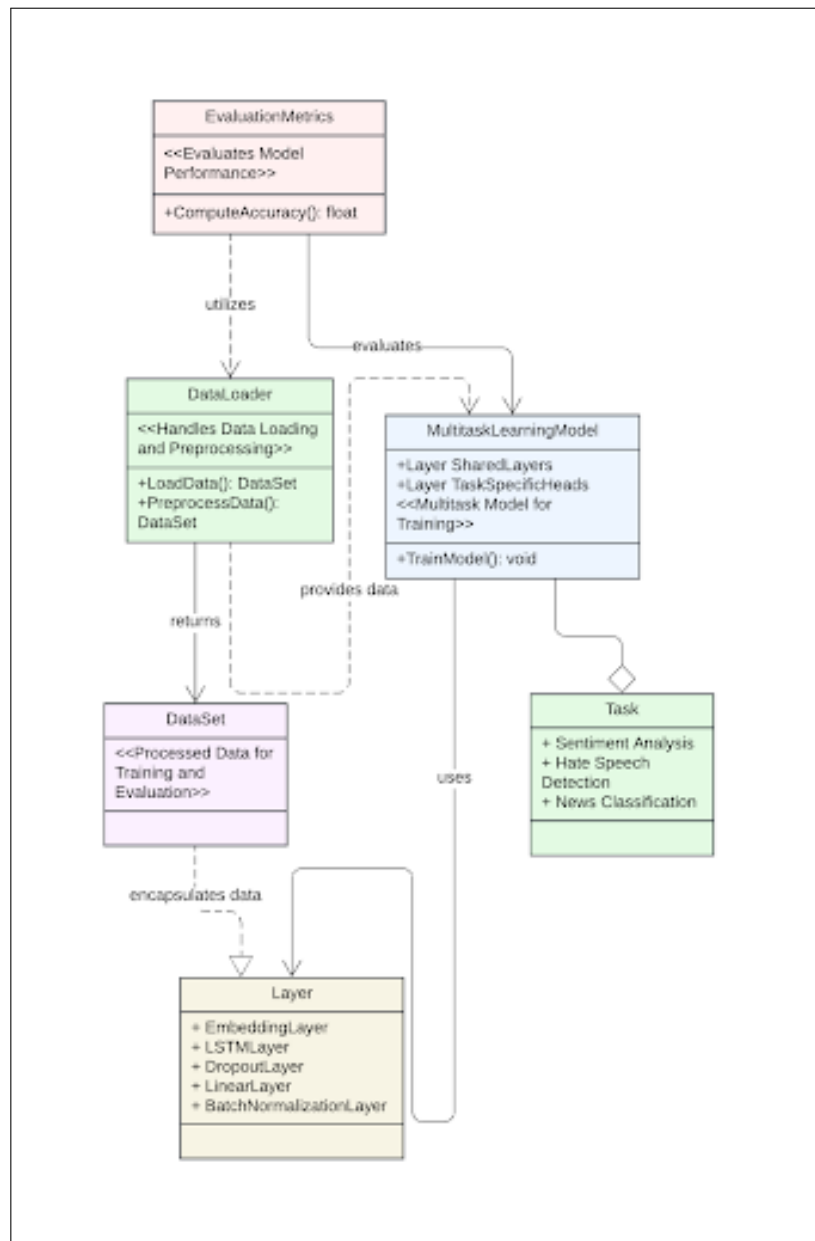


Figure 4.2: UML Class Diagram

Diagram 4.2 shows the structure of a multitask learning model system, including classes like `DataLoader`, `MultitaskLearningModel`, and `EvaluationMetrics`. The relationships indicate how data flows and is processed within the model, highlighting key functions like loading, training, and evaluating.

4.3 UML USE-CASE DIAGRAMS

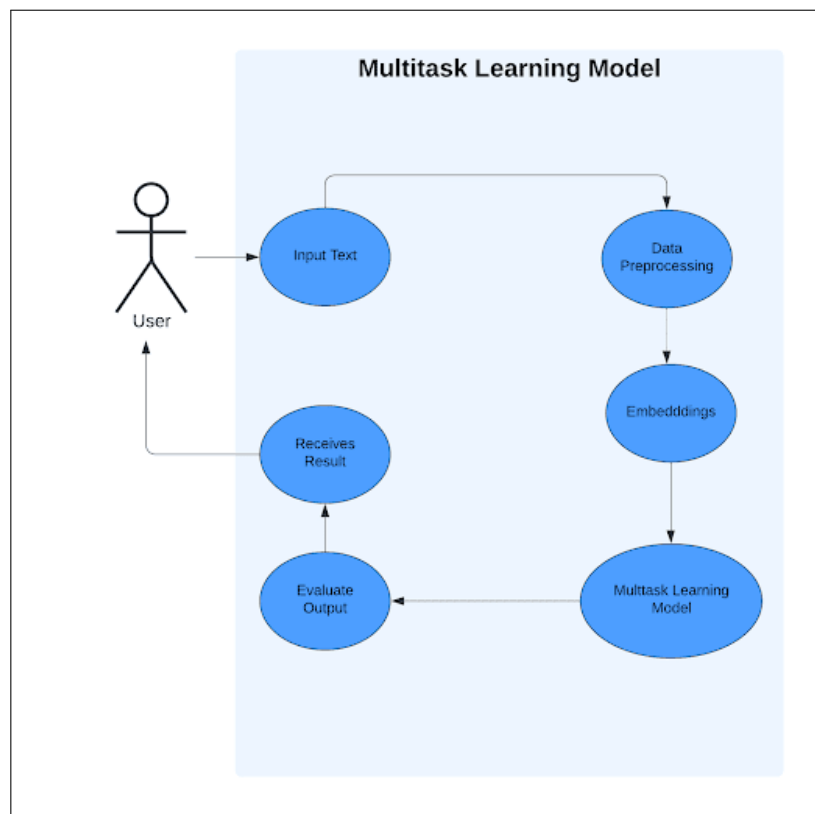


Figure 4.3: UML Use-Case Diagram

Diagram 4.3 depicts the interaction between the user and the multitask learning model, showcasing the steps from input text to output generation. Key processes like tokenization, embedding, and task-specific classifications are included to represent how different tasks are managed within the model.

4.4 UML ACTIVITY DIAGRAMS

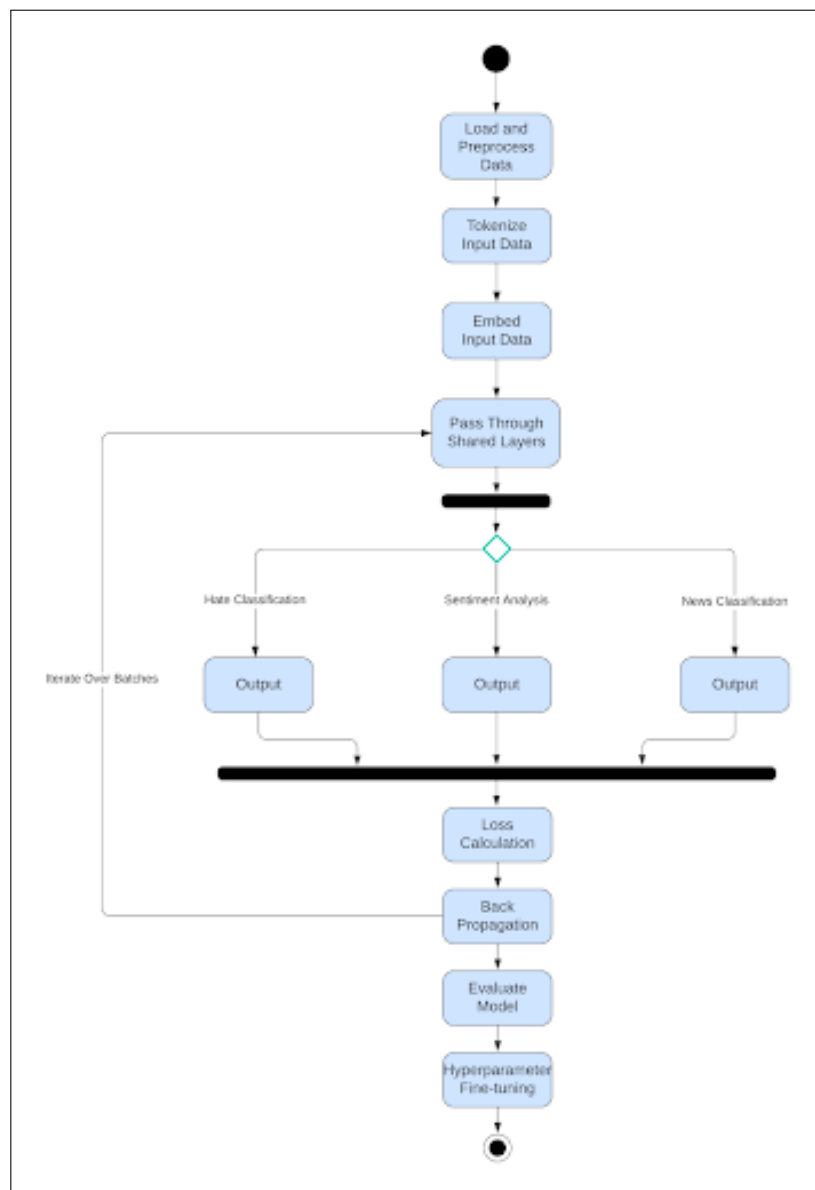


Figure 4.4: UML Activity Diagram

Diagram 4.4 visualizes the workflow of a multitask learning system from data preprocessing to output generation, evaluation, and hyperparameter fine-tuning. It outlines the sequential flow of tasks like hate classification, sentiment analysis, and backpropagation across multiple layers.

4.5 DATA FLOW DIAGRAM

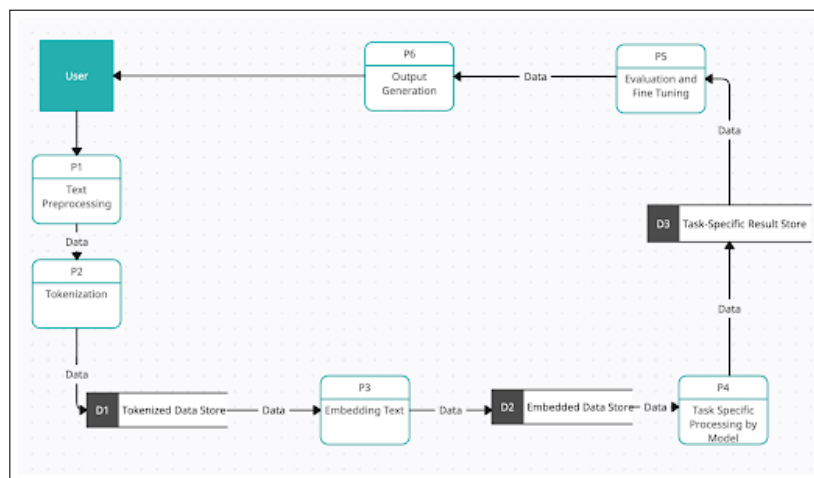


Figure 4.5: Data Flow Diagram

Diagram 4.5 outlines a text processing pipeline where input data undergoes preprocessing, tokenization, and embedding, followed by task-specific processing and evaluation. The final output is generated after fine-tuning, with intermediate results stored at each stage for task-specific usage.

4.6 SEQUENCE DIAGRAM

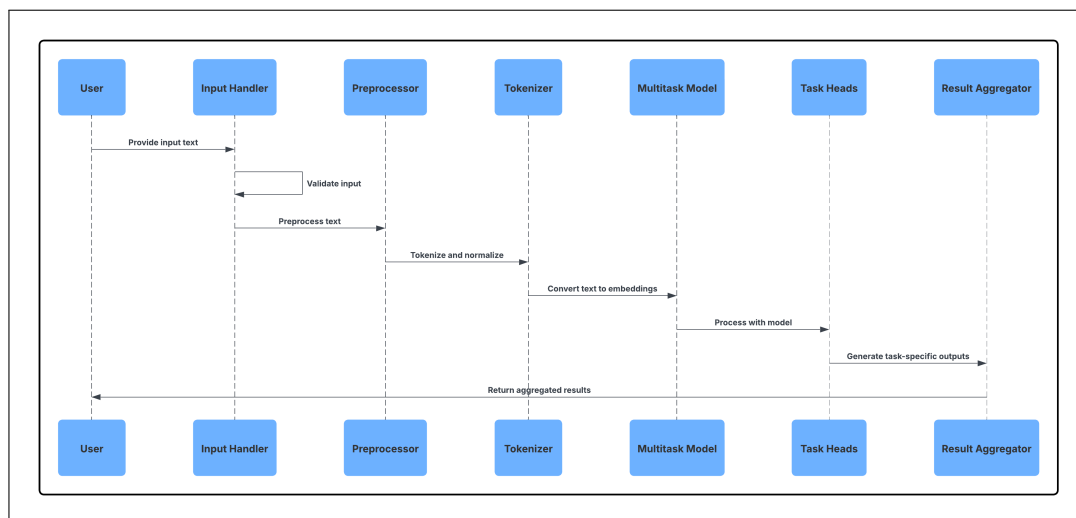


Figure 4.6: Sequence Diagram

Diagram 4.6 illustrates the flow of interactions within the system, from user input to task-specific output generation. It begins with the user providing text input, followed by preprocessing, tokenization, and embedding generation. The embeddings are processed by the multitask model, which routes them through shared and task-specific layers, generating results for tasks like Sentiment Analysis or Named Entity Recognition.

4.7 COMPONENT DIAGRAM

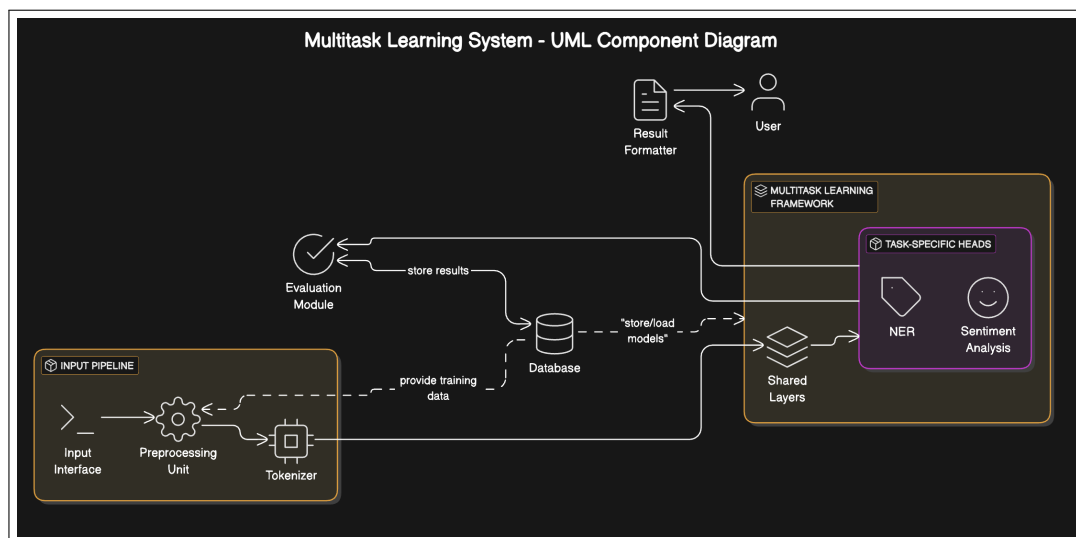


Figure 4.7: Component Diagram

The diagram 4.7 represents the architecture of the multitask learning framework. It includes modules like the Input Interface, Preprocessing Unit, Tokenizer, Multitask Model (shared and task-specific layers), and Evaluation Module. Each component interacts to handle input text, preprocess it, tokenize it, and generate task-specific outputs efficiently.

4.8 DEPLOYMENT DIAGRAM

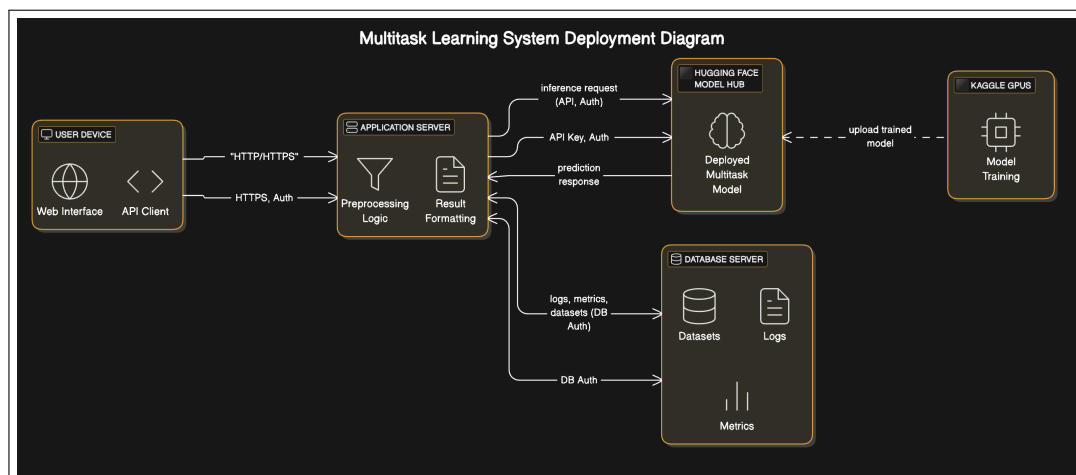


Figure 4.8: Deployment Diagram

The diagram 4.8 maps the multitask learning framework to its physical nodes. The model is trained on Kaggle GPUs and deployed on Hugging Face’s model hosting platform. Users access the deployed model through APIs or a web interface, while the preprocessing and result formatting happen locally or on the client-side. The system leverages Hugging Face’s infrastructure for efficient model inference and availability.

CHAPTER 5

PROJECT PLAN

5.1 PROJECT ESTIMATE

5.1.1 Reconciled Estimates

Research and Development – 6 Months

This phase includes an in-depth study of code-mixed data, exploration of current transformer-based NER models, and the development of a complete model pipeline. Key activities include dataset collection and preprocessing, fine-tuning of base and specialized models (e.g., BERT, IndicBERT, HingBERT), and implementation of evaluation metrics. The goal is to design a system capable of handling multilingual and informal text effectively, with a focus on improving entity recognition performance in real-world scenarios.

Testing and Evaluation – 2 Months

During this stage, the system will undergo extensive testing, including functional validation, model benchmarking, and performance analysis on unseen datasets. The evaluation will include metrics such as accuracy, precision, recall, and F1-score. Robustness checks will be performed to assess model behavior across noisy, ambiguous, and transliterated inputs. Feedback from peer reviews and faculty evaluations will be used to further refine the system.

Documentation and Review – 1.5 Months

The final phase will focus on creating comprehensive documentation, including user manuals, technical implementation reports, and system design descriptions. Feedback sessions with domain experts and academic mentors will be held to ensure alignment with the project objectives. This documentation will serve as a resource for future enhancements and support reproducibility for similar research in multilingual NLP tasks.

5.1.2 Project Resources

Human Resource

- Responsible for overseeing the overall progress.
- Ensuring effective communication between team members.
- Managing the timeline.
- Handling project documentation.
- Coordinating the implementation phases.

Hardware Resources

- 2 GPU/TPU machines for training the multitask learning models
- 1 Server for database management and model hosting.

Software Resources

- Python with TensorFlow/PyTorch for model development.
- FastAPI for backend services.
- Streamlit for UI.

Tools and Technologies

- **Programming:** Python

- **Libraries:** Transformers, PyTorch, Scikit-learn, Pandas, Matplotlib
- **Visualization:** Seaborn, Matplotlib, Excel
- **Collaboration:** GitHub, Google Docs, Meet.

5.2 RISK MANAGEMENT

5.2.1 Risk Identification

- **Data Availability:** Lack of sufficient labeled Marathi datasets for certain NLP tasks could affect model training and evaluation.
- **Model Overfitting:** The multitask model may suffer from overfitting, especially with limited data for some tasks.
- **Task Interference:** Task combinations that are not well-related may lead to negative interference, impacting overall model performance.
- **Hardware Constraints:** Insufficient access to high-performance computational resources (GPU/TPU) could lead to slow training and delays in model development.
- **Integration Issues:** Difficulty in integrating various components (model, API, UI) into a seamless system.

5.2.2 Risk Analysis

Likelihood

- **Data Availability:** Medium (due to the low-resource nature of Marathi NLP datasets).
- **Model Overfitting:** Medium (with appropriate regularization and validation techniques).
- **Task Interference:** Low (if proper task grouping strategies are followed).

- **Hardware Constraints:** Medium-High (depends on availability of computational resources).
- **Integration Issues:** Low (given the modular design of the system).

Impact

- **Data Availability:** High (could delay or limit the scope of the project).
- **Model Overfitting:** Medium (would require retraining and hyperparameter tuning).
- **Task Interference:** Medium (could impact specific task performance).
- **Hardware Constraints:** High (could delay development and model training).
- **Integration Issues:** Low (manageable with adequate planning).

5.2.3 Overview of Risk Mitigation, Monitoring, Management

To mitigate and manage potential risks throughout the development and deployment of the system, the following strategies will be implemented:

Mitigation

- **Data Availability:** Medium (due to the low-resource nature of Marathi NLP datasets).
- **Model Overfitting:** Medium (with appropriate regularization and validation techniques).
- **Task Interference:** Low (if proper task grouping strategies are followed).

- **Hardware Constraints:** Medium-High (depends on availability of computational resources).
- **Integration Issues:** Low (given the modular design of the system).

Monitoring Regular project status meetings, continuous performance tracking during model training, and issue tracking during system integration.

Management Establish a clear escalation process for any issues, with designated team members responsible for resolving specific risks.

5.3 PROJECT SCHEDULE

5.3.1 Project Task Set

1. Literature Survey and Dataset Collection
2. Data Cleaning and Annotation
3. Selection of Pre-trained Models
4. Model Training and Fine-tuning
5. Evaluation Metrics Setup
6. Comparative Model Analysis
7. Visualization of Results
8. Iterative Testing & Error Analysis
9. Model Optimization
10. Final Model Selection
11. System Design Diagrams (architecture, data flow)
12. Interface Design (if required for demo visualization)
13. Compilation of Results
14. Documentation of Experiments and Learnings
15. Project Report Writing
16. Final Testing and Review
17. Submission and Presentation Preparation

5.3.2 Task Network

- Task Dependencies
 - Dataset Collection → Preprocessing → Model Training
 - Model Training → Testing and Evaluation
 - Testing → System Integration
 - System Integration → Deployment
- This sequence ensures that tasks are performed logically and efficiently, allowing early testing of the model before full system integration. Dependencies are mapped out so that any delays in earlier phases do not affect later phases of the project.

5.3.3 Timeline Chart

- **Phase 1: Research and Literature Review (Sept-Oct)**
 - Study existing multitask learning frameworks and applications in low-resource NLP settings.
 - Identify datasets, models, and benchmarks for comparison.
- **Phase 2: Dataset Collection and Preprocessing (Nov-Dec)**
 - Gather datasets for both low-resource and auxiliary tasks.
 - Perform necessary preprocessing steps such as tokenization, normalization, and data augmentation where applicable.
- **Phase 3: Model Development and Selection (Dec-Jan)**
 - Develop the multitask learning framework.

- Implement the shared architecture for different tasks, integrating task-specific heads for each NLP task.
- Conduct initial experiments on task-relatedness.
- **Phase 4: Experimentation and Evaluation (Jan-Feb)**
 - Train models on collected datasets.
 - Evaluate the multitask learning model's performance against single-task learning and transfer learning.
 - Conduct experiments to explore the effect of task relatedness on performance.
- **Phase 5: Optimization and Fine-Tuning (Feb-Mar)**
 - Fine-tune hyperparameters to optimize model performance.
 - Conduct ablation studies to understand the impact of individual tasks on the overall framework.
- **Phase 6: Documentation and Reporting (April)**
 - Prepare detailed documentation, the final report, and presentation slides.
 - Submit findings and code for peer review.

5.4 TEAM ORGANIZATION

5.4.1 Team structure

- **Amogh Thakurkesai (Project Lead):** Responsible for overseeing the overall progress of the project, ensuring deadlines are met, and managing the project's scope. Amogh will also handle coordination between team members and ensure smooth communication throughout the project.

- **Abhay Shanbhag (Model Development & Evaluation):** Focuses on the development, training, and evaluation of the Multitask Learning (MTL) model. Abhay will handle the fine-tuning of the BERT-based model for various Marathi NLP tasks and evaluate the model's performance.
- **Ridhima Sinare (Backend Development):** Handles the backend development, including the creation of the API and database management. Ridhima will ensure the model's integration with the database and the proper functioning of the server-side logic.
- **Suramya Jadhav (Frontend Development):** Works on the user interface (UI), including features for dataset upload, viewing results, and interacting with the MTL model. Suramya will design and implement the frontend interface to ensure a seamless experience for the user.
- **Prof. S. S. Sonawane:** Internal mentor guiding the model selection, evaluation strategies, and overall research direction.
- **Dr. P. T. Kulkarni (Director, PICT):** Project advisor providing strategic oversight and academic alignment.

5.4.2 Management reporting and communication

- **Weekly Meetings:** The team will hold weekly meetings to discuss progress, challenges, and any adjustments needed to the project plan. Each team member will report on their specific tasks and goals for the upcoming week.
- **Communication Tools:** The team will use Slack for day-to-day communication and collaboration. This will allow for quick discussions, file sharing, and real-time updates on project tasks.
- **Project Management Tools:** A tool like Trello or Asana will be used to manage project tasks and track deadlines. This will help visualize task progress, assign responsibilities, and ensure that everyone is on the same page.

- **Documentation and Reporting:** The Project Lead will be responsible for preparing regular updates, progress reports, and ensuring proper documentation of the work. This includes maintaining the project timeline and handling any necessary communication with external stakeholders or mentors.
- **Collaboration and Feedback:** The team will collaborate closely during key phases such as model evaluation and integration, and provide feedback during the documentation and reporting stages. Continuous peer review will help ensure quality and accuracy across all components of the project.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 OVERVIEW OF PROJECT MODULES

The project is divided into several key modules to ensure systematic implementation and management. These modules include:

- **Data Collection and Preprocessing Module:** This module focuses on gathering datasets for various tasks such as sentiment analysis, hate speech detection, and news classification. It includes preprocessing steps such as tokenization, normalization, and data augmentation to prepare the data for model training.
- **Model Development and Training Module:** This module is responsible for developing the Multitask Learning (MTL) pipeline, which integrates BERT embeddings and task-specific classification heads for Marathi NLP tasks. It involves training the model on the collected datasets and evaluating its performance on multiple tasks simultaneously.
- **Evaluation and Experimentation Module:** This module evaluates the performance of the MTL model against single-task learning (STL) and transfer learning. It conducts experiments to test the effect of task-relatedness on model performance and fine-tunes the model as necessary.
- **User Interface Module:** This module provides a web-based user interface (UI) that allows users to upload datasets, view classification results, and monitor the performance of the model through various visualizations and metrics.
- **Backend and Database Module:** The backend handles the model integration and manages interactions with the database. This module ensures that the model results are stored and retrieved efficiently from the database, and it provides an API for communication between the frontend and the backend.

6.2 TOOLS AND TECHNOLOGIES USED

- **Programming Languages:**

- Python is used for model development, training, and evaluation. It provides extensive support for machine learning and natural language processing tasks.
- JavaScript (with React or Angular) is used for building the user interface.

- **Machine Learning Libraries:**

- PyTorch or TensorFlow is used for implementing the MTL model. These libraries are optimized for deep learning tasks and provide high-level abstractions for model development and training.
- Transformers (by Hugging Face) is used for utilizing pre-trained BERT models for feature extraction in Marathi.

- **Web Framework:**

- FastAPI is used for the backend development of the project. It allows for fast, efficient, and asynchronous communication between the frontend and backend.

These tools collectively enabled efficient development, experimentation, and analysis of the multilingual NER models tailored to handle code-mixed linguistic data.

6.3 ALGORITHM DETAILS

These algorithms are essential in implementing the multitask learning system, ensuring that the model is trained efficiently and that the most effective task combinations are identified for the best results.

Algorithm 1 Multitask Learning Model for Marathi NLP

- 1: **Overview:** This algorithm uses Multitask Learning (MTL) to train a model on multiple related NLP tasks simultaneously. The model utilizes BERT embeddings for feature extraction, which are then passed through task-specific heads for each classification task.
 - 2: **Input:** The system receives tokenized Marathi text for multiple tasks (sentiment analysis, hate speech detection, news classification).
 - 3: **Feature Extraction:** The BERT model extracts contextualized word embeddings for each task.
 - 4: **Task-Specific Heads:** Each task has its own classification head.
 - 5: **for** each task **do**
 - 6: For sentiment classification, the output is a sentiment label.
 - 7: For news classification, it outputs categories like sports, politics, etc.
 - 8: **end for**
 - 9: **Training:** The model is trained using a shared representation of the input data, which is optimized for all tasks using backpropagation.
 - 10: **Output:** The model provides task-specific predictions, and the performance is evaluated using metrics like accuracy, precision, and recall.
-

Algorithm 2 Task Combination Evaluation and Optimization

- 1: **Overview:** This algorithm evaluates different combinations of tasks in the multitask learning framework and optimizes task selection to improve model performance.
 - 2: **Input:** A set of predefined tasks (sentiment analysis, hate speech detection, news classification) with training data for each.
 - 3: **Task Combinations:** The algorithm evaluates all possible combinations of tasks (e.g., combinations of 2, 3, or 4 tasks) to identify the best-performing groupings.
 - 4: **Model Training:** The model is trained on each combination of tasks, and performance metrics are recorded.
 - 5: **Optimization:** Hyperparameters are tuned for each task combination to improve model performance.
 - 6: **Output:** The algorithm outputs the optimal task combination(s) that maximize the model's overall performance while minimizing task interference.
-

CHAPTER 7

SOFTWARE TESTING

7.1 TYPE OF TESTING

UNIT TESTING

Purpose: To test individual components of the multitask learning pipeline in isolation.

Application: We performed unit testing on various modules such as sentence tokenization, label formatting, and input-output transformation components for different subtasks.

Example: Ensuring the tokenizer handles Marathi script and special characters correctly; verifying correct label encoding for each task.

Tools: We used the framework `pytest` to test the correctness of helper functions and core utilities.

FUNCTIONAL TESTING

Purpose: To verify that the model's multitask outputs meet expected requirements.

Application: We evaluated the full pipeline on input sentences to verify outputs across all tasks — hate speech detection, sentiment analysis, news category classification, topic classification, and offensive content detection.

Example: Supplying sentences with known sentiments or hate speech indicators and verifying that the model correctly identifies them.

Tools: This was done using manually designed test cases that map to each task's expected behavior.

7.2 TEST CASES

Test Case 1

Sentence:

"हा लेख निवडणुकीच्या निकालांबद्दल आहे आणि लोकांचा सरकारवर राग व्यक्त करतो."

Task	Label
Hate Speech Detection	NOT
Sentiment Analysis	Negative
News Category Classification	Politics
Topic Classification	Politics
Offensive Content Detection	NOT

Test Case 2

Sentence:

"तुमच्यासारख्या लोकांनी देशाची नासधूस केली आहे."

Task	Label
Hate Speech Detection	HOF
Sentiment Analysis	Negative

News Category Classification	Crime
Topic Classification	Crime
Offensive Content Detection	OFEN

Test Case 3

Sentence:

पंतप्रधानांनी ग्रामीण विकासासाठी नवीन योजना जाहीर केली आहे.

Task	Label
Hate Speech Detection	NOT
Sentiment Analysis	Positive
News Category Classification	Politics
Topic Classification	Politics
Offensive Content Detection	NOT

Test Case 4

Sentence:

तू काहीच समजत नाहीस, सोशल मीडियावर काहीतरी बोलायचं तर आधी माहिती घे!

Task	Label
Hate Speech Detection	HOF
Sentiment Analysis	Neutral
News Category Classification	International

Topic Classification	International
Offensive Content Detection	OFEN

Test Case 5

.

Sentence:

सरकारी रुग्णालयात अजूनही रुग्णांसाठी पुरेशी व्यवस्था नाही.

Task	Label
Hate Speech Detection	NOT
Sentiment Analysis	Negative
News Category Classification	Health
Topic Classification	Health
Offensive Content Detection	NOT

CHAPTER 8

RESULTS

8.1 TEST AND EVALUATION RESULTS

This section evaluates the performance of multitask learning (MTL) on the five datasets—Hate-4c, Hate-2c, SHC, LPC, and Sent—for Marathi text classification. The results are presented for single-task learning (STL) and multitask learning configurations, including all combinations of 2, 3, 4, and 5 tasks simultaneously.

Dataset	STL	MTL
Hate-4c	56.25	60.94
Hate-2c	80.47	83.62
SHC	66.41	71.80
LPC	46.69	75.00
Sent	81.25	76.56

Table 8.1: Comparison Between STL and MTL Results

Table 8.1 directly compares the performance of Single Task Learning (STL) and Full Multitask Learning (MTL). Each dataset’s evaluation score under STL represents its performance when trained in isolation. In contrast, the Full MTL column represents the evaluation results of each dataset when trained alongside all other datasets in a shared multitask learning framework.

The results highlight the improvements gained through MTL, such as Hate-4c achieving a significant boost (60.94% vs. 56.25%) as well as LPC showing remarkable improvement (75.00% vs. 46.69%). Conversely, it also reveals instances where STL slightly outperforms MTL, such as with the Sent dataset (81.25% in STL vs. 76.56% in MTL), possibly indicating task interference. This comparison underscores the trade-offs inherent in multitask learning, particularly the balance between shared learning benefits and potential task conflicts.

Task Combination	Hate-4c	Sent	SHC	LPC	Hate-2c
Hate-4c, Sent, SHC, LPC	53.91	78.91	75.78	73.44	-
Hate-4c, Sent, SHC, Hate-2c	54.69	75.78	76.56	-	82.03
Hate-4c, Sent, LPC, Hate-2c	58.59	78.12	-	64.06	81.25
Hate-4c, SHC, LPC, Hate-2c	58.59	-	68.75	70.31	83.59
Sent, SHC, LPC, Hate-2c	-	82.03	75.78	62.50	85.16

Table 8.2: Performance for Four Tasks at a Time

Task Combination	Hate-4c	Sent	SHC	LPC	Hate-2c
Hate-4c, Sent, SHC	46.88	78.12	70.31	-	-
Hate-4c, Sent, LPC	54.69	78.91	-	65.62	-
Hate-4c, Sent, Hate-2c	57.02	77.34	-	-	74.22
Hate-4c, SHC, LPC	54.69	-	62.50	76.56	-
Hate-4c, SHC, Hate-2c	55.47	-	59.38	-	81.25
Hate-4c, LPC, Hate-2c	54.69	-	-	64.06	84.38
Sent, SHC, LPC	-	78.91	24.22	70.31	-
Sent, SHC, Hate-2c	-	80.47	73.44	-	79.69
Sent, LPC, Hate-2c	-	78.91	-	68.75	82.03
SHC, LPC, Hate-2c	-	-	76.56	76.56	87.50

Table 8.3: Performance for Three Tasks at a Time

8.1.0.1 Performance for Four Tasks at a Time

Table 8.2 explores the performance of each dataset when grouped with four tasks at a time. Each row represents a unique combination of four tasks, showing the results for each task in the grouping. It emphasizes how adding or removing a task impacts performance, offering insights into task-relatedness. For instance, Hate-4c achieves better performance in combinations involving LPC, while SHC improves with Hate-2c, highlighting synergistic effects between certain datasets.

8.1.0.2 Performance for Three Tasks at a Time

Table 8.3 delves deeper into three-task combinations, presenting individual results for each task within these groupings. It illustrates the trade-offs between performance improvement and degradation due to task interference. We observe that Hate-2c consistently

performs well across most three-task combinations, while SHC and Sent occasionally experience a drop in result when paired with certain tasks, reflecting potential conflicts in task representations.

8.1.0.3 Performance for Two Tasks at a Time

Table 8.4 evaluates the performance of datasets in combinations of two, thus evaluating task-specific synergies one-on-one. Pairwise results demonstrate how certain datasets, such as Hate-2c and LPC, achieve strong improvements due to complementary task features. However, tasks like SHC exhibit variability, indicating the nuanced interplay of shared and independent features between paired datasets.

Task Combination	Hate-4c	Sent	SHC	LPC	Hate-2c
Hate-4c, Sent	49.22	78.12	-	-	-
Hate-4c, SHC	52.34	-	62.50	-	-
Hate-4c, LPC	58.59	-	-	60.94	-
Hate-4c, Hate-2c	47.66	-	-	-	85.94
Sent, SHC	-	82.03	61.72	-	-
Sent, LPC	-	82.03	-	48.44	-
Sent, Hate-2c	-	79.69	-	-	87.50
SHC, LPC	-	-	57.03	72.66	-
SHC, Hate-2c	-	-	58.59	-	83.59
LPC, Hate-2c	-	-	-	60.16	86.72

Table 8.4: Performance for Two Tasks at a Time

Overall, we observe that Full MTL significantly improves Hate-4c (60.94% vs. 56.25% in STL) and LPC (75.00% vs. 46.69% in STL). However, for tasks like Sent and SHC, smaller task combinations (e.g. 2 or 3 tasks) often yield better results. Tasks with shared linguistic features (e.g., LPC and Hate-2c) achieve their highest performance in certain smaller combinations, such as LPC’s 76.56% with SHC and Hate-2c, and Hate-2c’s 87.50% with LPC and SHC, while pairwise and larger combinations reveal potential task interference. For instance, SHC performs poorly when combined with Sent and LPC (24.22%), likely due to conflicting features or representations.

Dataset-wise inference:

Hate-4c: Benefits most from full MTL and combinations with LPC.

Hate-2c: Consistently performs well across all combinations, peaking in pairwise tasks.

SHC: Shows improved performance with LPC and Hate-2c but struggles with unrelated tasks.

LPC: Demonstrates strong gains in multitask settings, particularly with SHC and Hate-2c.

Sent: Performs best in pairwise and 4-task combinations, indicating it leverages task-specific synergies.

The results highlight the importance of carefully selecting tasks in MTL settings. While full MTL offers robustness and generalization, targeted combinations can yield better task-specific results, depending on the degree of task relatedness.

8.2 SCREEN SHOTS

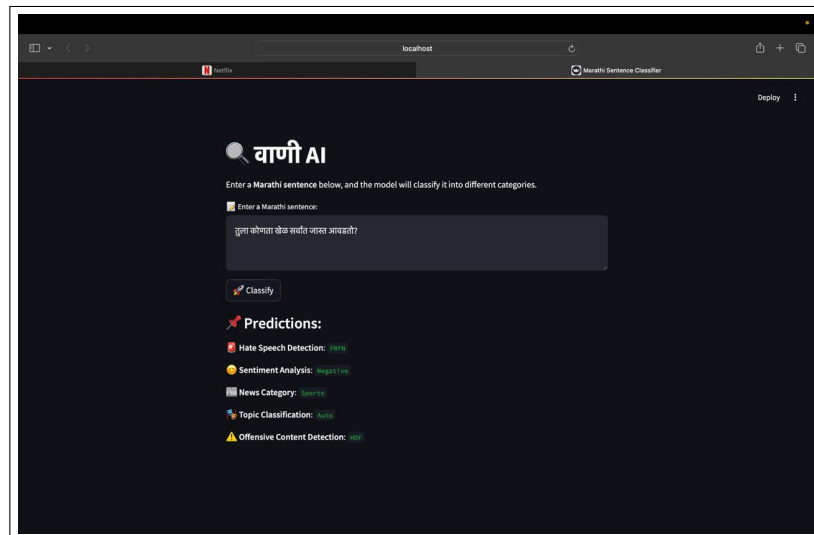


Figure 8.1: UI for Multi Task Learning Framework

CHAPTER 9

CONCLUSIONS

9.1 CONCLUSIONS

The project successfully implemented a Multitask Learning (MTL) framework for Marathi NLP tasks, specifically focusing on sentiment analysis, hate speech detection, and news classification. The key findings from this research are: **Improvement with Multitask Learning:** The use of MTL allowed for the sharing of learned representations across different NLP tasks, leading to enhanced generalization and improved performance in tasks with limited labeled data, especially for Marathi, a low-resource language.

Task-Relatedness Impact: The study showed that task-relatedness significantly influences the model's performance. Careful selection and grouping of related tasks produced better results, while mismatched tasks led to task interference and decreased accuracy.

Model Performance: The MTL model outperformed single-task learning (STL) in tasks like Hate Speech Detection and News Classification, demonstrating the effectiveness of multitask learning for low-resource language applications.

Scalability: The system is scalable and can be extended to include more tasks or datasets, enhancing its applicability to other low-resource languages.

User-Friendly Interface: The development of an intuitive user interface allowed non-experts to interact with the system effectively, upload datasets, and view model results.

9.2 FUTURE WORK

Expansion to Other Low-Resource Languages: The model can be adapted and extended to other low-resource languages, such as Hindi, Tamil, and Kannada, by following the same framework and incorporating language-specific preprocessing and task selection.

Additional NLP Tasks: Future work can involve incorporating more advanced NLP tasks such as Named Entity Recognition (NER), Machine Translation, and Text Summarization to improve the system's versatility and overall utility.

Improved Model Architecture: Exploring advanced MTL architectures, such as multi-level attention mechanisms or graph-based models, could further improve task-relatedness handling and model performance.

Integration of Unsupervised Learning: To address data scarcity, integrating unsupervised learning techniques and semi-supervised learning could reduce dependency on labeled data and make the model more adaptable to real-world data.

Real-time Applications: Future work can focus on integrating this framework into real-time applications such as social media monitoring and content moderation systems.

9.3 APPLICATIONS

Sentiment Analysis in Social Media: The developed system can be used to analyze sentiment in Marathi social media content, helping businesses and governments understand public opinion and social trends.

Hate Speech Detection: The framework can be applied to monitor online platforms for harmful content, enabling faster detection and response to cyberbullying, hate speech, or offensive content in Marathi.

News Categorization: The system can be used in news aggregators or news websites to automatically categorize Marathi news articles into predefined categories, improving content discovery and user experience.

Cross-Lingual NLP Systems: By adapting the model for other Indian languages, the system can be used as a foundation for building multilingual and cross-lingual NLP applications for Indian languages, enabling better accessibility to digital content.

Educational Tools: The system can be integrated into educational platforms to automatically classify and analyze texts, providing insights into content readability, sentiment, and emotion in educational materials.

CHAPTER 10

APPENDIX

APPENDIX A

Feasibility Assessment of Multitask Learning for Marathi Text Classification using Shared Representations and Computational Complexity

1. INTRODUCTION

Multitask Learning (MTL) has emerged as a powerful technique in Natural Language Processing (NLP) due to its ability to learn multiple related tasks simultaneously. This feasibility assessment examines the application of MTL to Marathi text classification tasks, such as sentiment analysis, hate speech detection, and news classification. By sharing representations across tasks, MTL aims to improve model performance, especially in low-resource settings, where data scarcity poses a significant challenge. In this assessment, we evaluate the computational difficulty of applying MTL to Marathi text classification using mathematical frameworks from complexity theory.

2. PROBLEM STATEMENT

Let the input to the system be a sequence of tokens $T = \{t_1, t_2, \dots, t_n\}$ representing a Marathi sentence, and let the goal be to assign each token t_i a label $l_i \in L$, where L is the set of task-specific tags (e.g., sentiment labels, hate speech categories, or news categories).

The goal is to determine whether there exists a function $f : T \rightarrow L$ such that the overall labeling maximizes the shared learning benefits across multiple tasks, while maintaining task-specific accuracy.

3. SATISFIABILITY FRAMEWORK

This multitask learning setup can be transformed into a constraint satisfaction problem (CSP) where each task shares common parameters, and constraints enforce consistency across tasks. Define a Boolean formula Φ such that:

$$\Phi = \bigwedge_{i=1}^n \left(\bigvee_{l \in L} x_{i,l} \right) \wedge \bigwedge_{\text{constraints}} C(x_{i,l}, x_{j,l'})$$

where $x_{i,l}$ is true if token t_i is assigned label l , and C encodes the constraints shared by all tasks, such as ensuring that task-specific features do not conflict during training.

Determining the satisfiability of Φ in the context of MTL is akin to solving the classical **Boolean Satisfiability Problem (SAT)**, which is **NP-Complete**.

4. COMPLEXITY CLASSIFICATION

Given the NP-completeness of SAT, MTL for Marathi text classification inherits similar complexity characteristics. The problem can be classified as follows:

- If the tasks are sufficiently related and task-specific outputs are minimized (e.g., using a shared hidden layer with individual classification heads for each task), the problem can be solved in **polynomial time (P)** using deep learning models like BERT, which approximate solutions through gradient-based methods.
- If the tasks involve long-range dependencies (e.g., understanding context in sentiment analysis or recognizing complex entities in news classification), the problem becomes **NP-Hard**, and solving it optimally is computationally intractable.

5. ALGEBRAIC MODELING

From an algebraic perspective, consider each token t_i as an element of a set T , and let the labeling function f be defined over an algebraic structure $(L, \circ)$, where $\circ$ represents the shared learning operation between tasks. The entire MTL process can be viewed as a function:

$$f : T^n \rightarrow L^n$$

subject to constraints that are defined through homomorphisms preserving linguistic relations across tasks. This algebraic structure aligns with the composition of shared layers and task-specific output layers in MTL models, similar to the principles of finite automata used for sequence labeling.

6. CONCLUSION

Multitask Learning (MTL) for Marathi text classification tasks, such as sentiment analysis, hate speech detection, and news classification, shows promising potential for enhancing model performance in low-resource settings. While MTL offers computational benefits by sharing representations across tasks, the general problem remains **NP-Hard** when full task interdependencies are considered.

NP-Hard in the general case, but **solvable in practice** using deep learning methods that approximate optimal solutions efficiently within polynomial time.

APPENDIX B

1. SURVEY PAPER DETAILS

Paper Status: Under Review

Name of Conference: ACL 2025

Paper Title: A Survey on Multi-Task Learning Paradigms for Low-Resource NLP

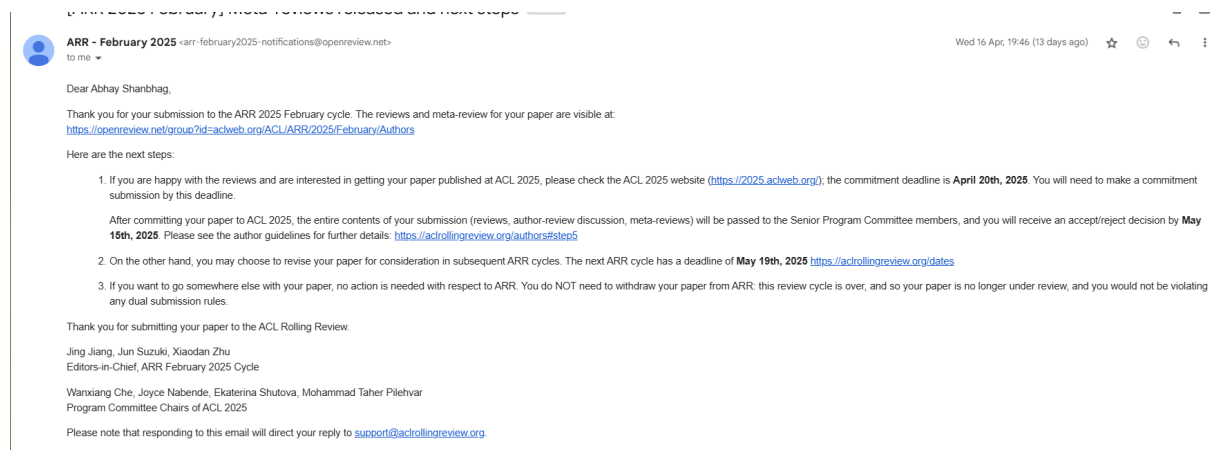


Figure 10.1: Submission Details

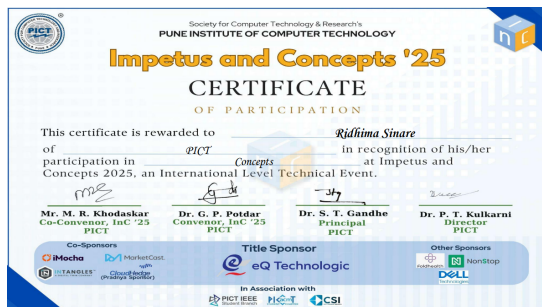
2. IMPLEMENTATION PAPER DETAILS

Paper Status: Under Review by L3 Cube Labs

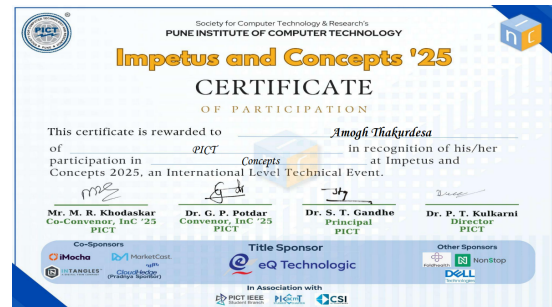
Name of Conference: NA

Paper Title: Enhancing Low Resource Languages using Multi Task Learning

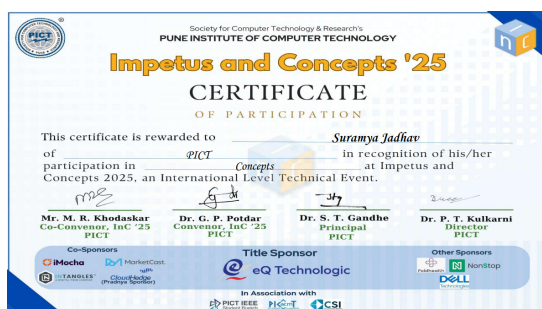
3. EVENT PARTICIPATION DETAILS



(a) Ridhima INC Certificate



(b) Amogh INC Certificate



(c) Suramya INC Certificate



(d) Abhay INC Certificate

Figure 10.2: INC Certificates

APPENDIX C

1. PLAGARISM REPORT

Document Information

Analyzed document	Group_22_BE_Project_Final_Report.pdf (D210178898)
Submitted	2025-04-26 06:56:00 UTC+02:00
Submitted by	
Submitter email	sssonawane@pict.edu
Similarity	10%
Analysis address	sssonawane.pict@analysis.urkund.com

Sources included in the report

W	URL: https://www.Bartleby.com/topics/sensitivity-analysis/41 Fetched: 2025-04-26 06:57:00		1
W	URL: https://www.coursehero.com/file/83472522/PROJECT-REPORT-1pdf/ Fetched: 2025-04-26 06:56:00		1
W	URL: https://ijariie.com/AdminUploadPdf/SERIAL_MULTIMODAL_BIOMETRIC_AUTHENTICATION_FOR_SECURITY_ijariie7575.pdf Fetched: 2025-04-26 06:57:00		1
W	URL: https://WWW.coursehero.com/file/236179261/BlckBookReport-Finalpdf/ Fetched: 2025-04-26 06:56:00		1
W	URL: https://aclanthology.org/2022.argmining-1.pdf Fetched: 2025-04-26 06:56:00		3
W	URL: https://www.zhuanzhi.ai/document/4a7131f6e358825301a790c62c60efa3 Fetched: 2025-04-26 06:56:00		4
W	URL: https://anoopkunchukuttan.gitlab.io/publications/acm_csur_2020_mnmt_survey.pdf Fetched: 2025-04-26 06:56:00		5
W	URL: https://assets-eu.researchsquare.com/files/rs-727385/v1_covered.pdf?c=1655920052 Fetched: 2025-04-26 06:57:00		1
W	URL: http://www.lrec-conf.org/proceedings/lrec-coling-2024/pdf/2024.main-1.529.pdf Fetched: 2025-04-26 06:57:00		1
W	URL: https://aclanthology.org/2025.chipsal-1.pdf Fetched: 2025-04-26 06:57:00		2

Figure 10.3: Plagiarism Report

CHAPTER 11

REFERENCES

- [1] Héctor Martínez Alonso and Barbara Plank. 2016. Multitask learning for semantic sequence prediction under varying data conditions. CoRR, abs/1612.02251.
- [2] Yangbin Chen, Yun Ma, Xudong Mao, and Qing Li. 2019. Multi-task learning for abstractive and extractive summarization. *Data Science and Engineering*, 4:14–23.
- [3] Michael Crawshaw. 2020. Multi-task learning with deep neural networks: A survey. ArXiv, abs/2009.09796.
- [4] Daxiang Dong, Hua Wu, W. He, Dianhai Yu, and Haifeng Wang. 2015a. Multi-task learning for multiple language translation. In *Annual Meeting of the Association for Computational Linguistics*.
- [5] Daxiang Dong, Hua Wu, Wei He, Yu Dianhai, and Haifeng Wang. 2015b. Multi-task learning for multiple language translation. Volume 1, pages 1723–1732. Association for Computational Linguistics.
- [6] Orhan Firat, Kyunghyun Cho, and Yoshua Bengio. 2016. Multi-way, multi-lingual neural machine translation with a shared attention mechanism. ArXiv, abs/1601.01073.
- [7] Lin Gui, Leng Jia, Jiyun Zhou, Ruifeng Xu, and Yulan He. 2020. Multi-task mutual learning for joint sentiment classification and topic detection. *IEEE Transactions on Knowledge and Data Engineering*, 34:1915–1927.
- [8] Han Guo, Ramakanth Pasunuru, and Mohit Bansal. 2018. Soft layer-specific multi-task summarization with entailment and question generation. ArXiv, abs/1805.11004.
- [9] Thanh-Le Ha, Jan Niehues, and Alexander H. Waibel. 2016. Toward multi-lingual neural machine translation with universal encoder and decoder. ArXiv, abs/1611.04798.
- [10] Adeep Hande, Siddhanth U Hegde, and Bharathi Raja Chakravarthi. 2022. Multi-task learning in under-resourced Dravidian languages. *Journal of Data, Information and Management*, 4:137–165.

- [11] Fariz Ikhwantri, Samuel Louvan, Kemal Kurniawan, Bagas Abisena, Valdi Rachman, Alfian Farizki Wicaksono, and Rahmad Mahendra. 2018. Multi-task active learning for neural semantic role labeling on low-resource conversational corpus. In *Proceedings of the Workshop on Deep Learning Approaches for Low-Resource NLP*, pages 43–50, Melbourne. Association for Computational Linguistics.
- [12] Melvin Johnson, Mike Schuster, Quoc V. Le, Maxim Krikun, Yonghui Wu, Zhifeng Chen, Nikhil Thorat, Fernanda Viégas, Martin Wattenberg, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2017. Google’s multilingual neural machine translation system: Enabling zero-shot translation. *Transactions of the Association for Computational Linguistics*, 5:339–351.
- [13] E. Kiperwasser and Miguel Ballesteros. 2018. Scheduled multi-task learning: From syntax to translation. *Transactions of the Association for Computational Linguistics*, 6:225–240.
- [14] Tom Kocmi and Ondrej Bojar. 2018. Trivial transfer learning for low-resource neural machine translation. In *Conference on Machine Translation*.
- [15] Atharva Kulkarni, Meet Mandhane, Manali Likhitkar, Gayatri Kshirsagar, and Raviraj Joshi. 2021. L3cubemahasent: A Marathi tweet-based sentiment analysis dataset. In *Proceedings of the Eleventh Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*, pages 213–220.
- [16] Ying Lin, Shengqi Yang, Veselin Stoyanov, and Heng Ji. 2018. A multi-lingual multi-task architecture for low-resource sequence labeling. In *Annual Meeting of the Association for Computational Linguistics*.
- [17] Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. 2016. Recurrent neural network for text classification with multi-task learning. Pages 2873–2879. AAAI Press.
- [18] Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. 2017. Adversarial multi-task learning for text classification. In *Annual Meeting of the Association for Computational Linguistics*.

- [19] A. Madhavaraj and A. G. Ramakrishnan. 2019. Data-pooling and multi-task learning for enhanced performance of speech recognition systems in multiple low-resourced languages. 2019 National Conference on Communications (NCC), pages 1–5.
- [20] Ahmed Magooda, Diane Litman, and Mohamed Elaraby. 2021. Exploring multitask learning for low-resource abstractive summarization. In Findings of the Association for Computational Linguistics: EMNLP 2021, pages 1652–1661, Punta Cana, Dominican Republic. Association for Computational Linguistics.
- [21] Chengzhi Mao, Amogh Gupta, Vikram Nitin, Baishakhi Ray, Shuran Song, Junfeng Yang, and Carl Vondrick. 2020. Multitask learning strengthens adversarial robustness. In Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II 16, pages 158–174. Springer.
- [22] Mounika Marreddy, Subba Reddy Oota, Lakshmi Sireesha Vakada, Venkata Charan Chinni, and Radhika Mamidi. 2022. Multi-task text classification using graph convolutional networks for large-scale low-resource language.
- [23] Bryan McCann, Nitish Shirish Keskar, Caiming Xiong, and Richard Socher. 2018. The natural language decathlon: Multitask learning as question answering. ArXiv, abs/1806.08730.
- [24] Quentin Meeus, Marie-Francine Moens, and Hugo Vanhamme. 2022. Multitask learning for low-resource spoken language understanding. Volume abs/2211.13703.
- [25] Aishwarya Mirashi, Srushti Sonavane, Purva Lingayat, Tejas Padhiyar, and Raviraj Joshi. 2024. L3cube-indicnews: News-based short text and long document classification datasets in Indic languages. arXiv preprint arXiv:2401.02254.
- [26] Saloni Mittal, Vidula Magdum, Sharayu Hiwarkhedkar, Omkar Dhekane, and Raviraj Joshi. 2023. L3cube-mahanews: News-based short text and long document classification datasets in Marathi. In International Conference on Speech and Language Technologies for Low-resource Languages, pages 52–63. Springer.
- [27] Jan Niehues and Eunah Cho. 2017. Exploiting linguistic resources for neural machine translation using multitask learning. In Conference on Machine Translation.

- [28] Shantipriya Parida and Petr Motlíček. 2019. Abstract text summarization: A low-resource challenge. In *Conference on Empirical Methods in Natural Language Processing*.
- [29] Hrushikesh Patil, Abhishek Velankar, and Raviraj Joshi. 2022. L3cube-mahahate: A tweet-based Marathi hate speech detection dataset and BERT models. In *Proceedings of the Third Workshop on Threat, Aggression and Cyberbullying (TRAC 2022)*, pages 1–9.
- [30] Aabha Pingle, Aditya Vyawahare, Isha Joshi, Rahul Tangsali, and Raviraj Joshi. 2023. L3cube-mahasent-md: A multi-domain Marathi sentiment analysis dataset and transformer models. In *Proceedings of the 37th Pacific Asia Conference on Language, Information and Computation*, pages 274–281.
- [31] Víctor M. Sánchez-Cartagena, Miquel Esplà-Gomis, Juan Antonio Pérez-Ortiz, and Felipe Sánchez-Martínez. 2021. Rethinking data augmentation for low-resource neural machine translation: A multitask learning approach. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8502–8516, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- [32] Gaurish Thakkar, Nives Mikelic Preradovic, and Marko Tadic. 2022. Multi-task learning for cross-lingual sentiment analysis. In *CLEOPATRA@WWW*.
- [33] Yiren Wang, Chengxiang Zhai, and Hany Hassan Awadalla. 2020. Multi-task learning for multilingual neural machine translation. *ArXiv*, abs/2010.02523.
- [34] Joseph Worsham and Jugal Kumar Kalita. 2020. Multi-task learning for natural language processing in the 2020s: where are we going? *ArXiv*, abs/2007.16008.
- [35] Poorya Zareemoodi and Gholamreza Haffari. 2018. Neural machine translation for bilingually scarce scenarios: a deep multi-task learning approach. In *North American Chapter of the Association for Computational Linguistics*.
- [36] Zhihan Zhang, W. Yu, Mengxia Yu, Zhichun Guo, and Meng Jiang. 2022. A survey of multi-task learning in natural language processing: Regarding task relatedness and training methods. *ArXiv*, abs/2204.03508.

- [37] Barret Zoph, Deniz Yuret, Jonathan May, and Kevin Knight. 2016. Transfer learning for low-resource neural machine translation. ArXiv, abs/1604.02201.

LIST OF ABBREVIATIONS

Abbreviation	Illustration
MTL	Multitask Learning
BERT	Bidirectional Encoder Representation from Transformer
NLP	Natural Language Processing
STL	Singletask Learning

LIST OF FIGURES

Figure	Illustration	Page No.
4.1	System Architecture	25
4.2	UML Class Diagram	26
4.3	UML Use-Case Diagram	27
4.4	UML Activity Diagram	28
4.5	DFD Level 0	29
4.6	Sequence Diagram	30
4.7	Component Diagram	31
4.8	Deployment Diagram	32
8.1	UI for MTL	57

LIST OF TABLES

Table	Illustration	Page No.
8.1	Comparison between STL and MTL	54
8.2	Performance for 4 tasks at a time Error	55
8.3	Performance for 3 tasks at a time Configuration	55
8.4	Performance for 2 tasks at a time Configuration	56